

We are going to discuss how to make a plot that does not involve simple graphical primitives- like rects, circles- but complex lines or shapes.

Line charts

Up until now, we have mostly made charts with graphical primitives in svg- like rects and circles- or simple lines. Many charts feature more complex shapes consisting in a path of points- like line charts. How can we create line charts using d3?

Lines and paths:

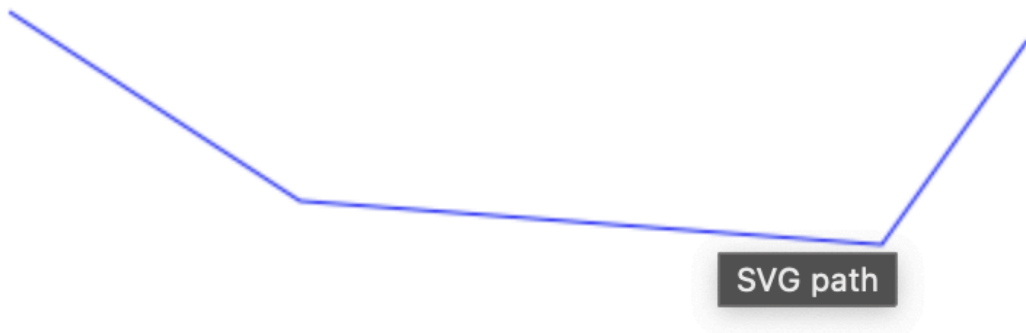
We are going to use the svg path element, more details here:

https://www.w3schools.com/graphics/svg_path.asp

The way a path element works is by defining a set of points that make up the path, in a specific format. These read as a set of step-by step instructions for creating the path. The path begins with 'M' followed by a pixel location, meaning 'move to' that location to begin the shape. Then, from the place you have moved, you can specify where to draw a line to using 'L' followed by a pixel location. The next line, will go from the previous line end point to the next defined end point, and so on. 'Z' can be used to close the path you have created.

Here's an example, first in pure html:

```
<svg width="500" height="200">
  <path style="fill: none; stroke: blue" d="M0 10 L100 75 L300 90 L350
20"></path>
</svg>
```



This is somewhat complex, so d3 tries to simplify things by using a line (and area) function :

```
const data = [{x: 0, y: 10}, {x: 100, y: 75}, {x: 300, y: 90}, {x: 350, y: 20}]
```

```
// Prepare a helper function
```

```
const line = d3.line()
```

```
  .x(d => d.x)
```

```
  .y(d => d.y);
```

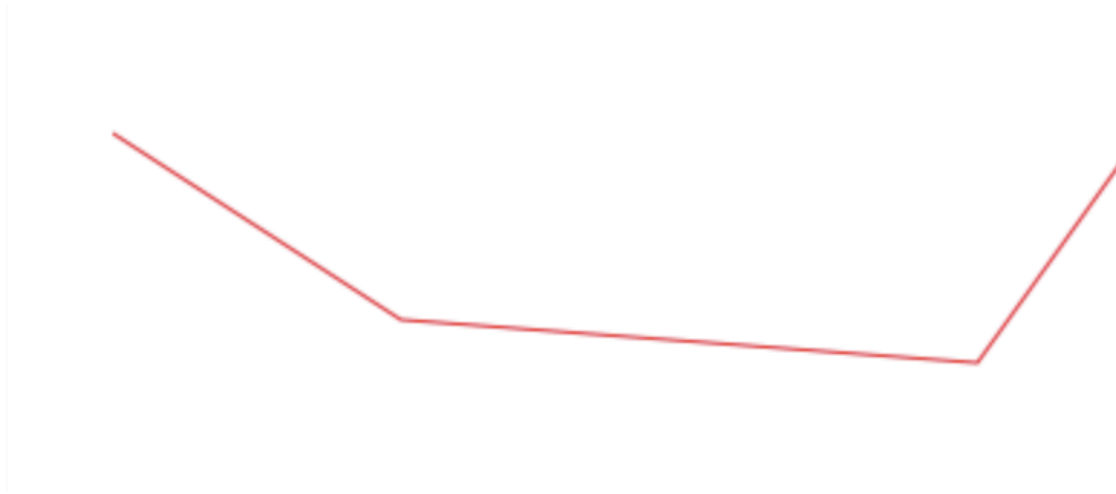
```
// Add the <path> to the <svg> container using the helper function
```

```
d3.select('svg').append('path')
```

```
  .attr('d', line(data))
```

```
  .attr('stroke', 'red')
```

```
  .attr('fill', 'none');
```



What if you wanted to fill the shape- like an area chart? D3 provides an area function. It needs both a definition of the top of the shape, as well as the bottom of the shape, so it knows where to fill in the color you have provided :

```
const data = [{x: 0, y: 10}, {x: 100, y: 75}, {x: 300, y: 90}, {x: 350, y: 20}]
```

```
// Prepare a helper function
const area = d3.area()
  .x(d => d.x)      // Same x-position
  .y1(d => d.y)     // Top line y-position
  .y0(150)          // Bottom line y-position

// Add the area path using this helper function
d3.select('svg').append('path')
  .attr('d', area(data))
  .attr('stroke', 'green')
  .attr('fill', 'green');
```



What if we want a more curvy shape? d3 has interpolators to compute a curve between endpoints. Careful- the selection of an interpolator can impact the presentation of the data and potentially distort or mislead. Use with caution.

```
const area = d3.area()  
  .x(d => d.x)  
  .y1(d => d.y)  
  .y0(150)  
  .curve(d3.curveNatural);
```



There is also a built in step function interpreter,

```
const line = d3.line()  
  .x(d => d.x)  
  .y(d => d.y)  
  .curve(d3.curveStep);
```



You can try out different curve interpreters here:

<http://bl.ocks.org/d3indepth/b6d4845973089bc1012dec1674d3aff8>

Putting it together with a chart:

S&P 500 Index



Suppose we assign the x and y parameters in our initialization function:

```
vis.xValue = d => d.date;  
vis.yValue = d => d.close;
```

Then assume we setup the svg and assign a group to this svg called 'chart'.

```
vis.svg = d3.select(vis.config.parentElement)  
    .attr('width', vis.config.containerWidth)  
    .attr('height', vis.config.containerHeight);  
  
    // Append group element to contain our actual chart w/margins  
    vis.chart = vis.svg.append('g')  
        .attr('transform',  
            `translate(${vis.config.margin.left},${vis.config.margin.top})`);
```

This chart will use a time scale for the x axis and a linear scale for the y axis:

```
vis.xScale = d3.scaleTime()  
    .domain(d3.extent(vis.data, vis.xValue))  
    .range([0, vis.width]);  
  
vis.yScale = d3.scaleLinear()
```

```
.domain(d3.extent(vis.data, vis.yValue));  
.range([vis.height, 0])  
.nice(); //helps the axis behave nicely
```

Your helper function can use the scales like this:

```
vis.line = d3.line()  
  .x(d => vis.xScale(vis.xValue(d)))  
  .y(d => vis.yScale(vis.yValue(d)));
```

Then, you can add your path to the svg like this:

```
// Add line path  
vis.chart.append('path')  
  .data([vis.data])  
  .attr('class', 'chart-line')  
  .attr('d', vis.line)  
  .attr('fill', 'none')  
  .attr('stroke', 'blue');
```

Here's a full example:

<https://codesandbox.io/s/github/UBC-InfoVis/2021-436V-examples/tree/master/d3-static-area-chart?file=/js/areachart.js>

Credits:

Adapted from

https://github.com/UBC-InfoVis/2021-436V-tutorials/tree/master/2_D3_Tutorial#making-line-and-area-charts